Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

Chapter 1

Introduction

1.1 Problem statement and societal relevance

The current trend in machine learning is moving towards increasingly large models and datasets [1]. Although deep learning approaches often achieve the highest predictive accuracy, their energy demand and associated CO_2 emissions increase disproportionately [2, 3]. In an era marked by strict sustainability guidelines and rising energy costs, a critical question arises: At what point is a computationally "simple" model ecologically more viable than a highly complex one?

The primary objective of this thesis is to determine whether and when it is worthwhile to deploy more complex models, considering not only their predictive accuracy but also their associated energy consumption. To achieve this, a Logistic Regression, a Random Forest, an XGBoost model and a Multilayer Perceptron are compared under equal conditions across three datasets of varying sizes. This comparative analysis aims to provide a clearer understanding of the thresholds at which simpler models represent the better choice. Additionally, the study investigates how modifying a Multilayer Perceptron architecture, specifically varying the number of neurons and hidden layers, directly impacts energy consumption and CO_2 emissions.

To address the problem statement and achieve the outlined objectives, this thesis investigates the overarching question: *How does the relationship between energy consumption and predictive accuracy change for Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron when model complexity and dataset size are systematically varied?*

Specifically, the following sub-questions are examined:

- How do energy consumption (in kWh) and CO_2 emissions change when scaling a deep learning architecture (variation of neuron count)?
- Under which conditions (dataset size, model complexity) does the resource consumption of complex models exceed the achievable accuracy gain compared to simpler algorithms?

- How does the energy consumption differ between CPU-based training (Random Forest) and GPU-accelerated training (XGBoost, MLP) for the same classification task?

1.2 Motivation: When does a simpler model pay off?

1.3 Research questions and objectives

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows: Chapter 2 provides the theoretical background, distinguishing between Green AI and Red AI, and introducing the relevant sustainability and classification metrics. Chapter 3 details the methodology, including dataset selection and the experimental hardware setup. Chapter 4 outlines the implementation process, focusing on data preprocessing, model training, and the logging of resource consumption. The empirical results are analyzed in Chapter 5, evaluating both performance and resource utilization across scaling dataset sizes. Chapter 6 discusses the findings, highlighting the economic and environmental implications. Finally, Chapter 7 concludes the thesis and provides actionable recommendations for sustainable machine learning practices.

$$S = \frac{F_1}{1 + \lambda_1 \cdot t_{\text{scaled}} + \lambda_2 \cdot c_{\text{scaled}}}$$

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations required to understand the empirical study presented in the remainder of this thesis. It first frames the research within the broader debate between *Red AI* and *Green AI*, then formalizes the concepts of energy consumption and carbon emissions in machine learning. Subsequently, the relevant classification algorithms are described from a purely theoretical standpoint, followed by the metrics used to evaluate their predictive performance, the validation strategy, and hyperparameter optimization techniques. All implementation details are deliberately deferred to Section 5.3.

2.1 Green AI and Red AI

This section introduces the conceptual distinction between Red AI and Green AI, which provides the vocabulary for framing the ecological evaluation of machine learning models and motivates the treatment of efficiency as a first-class evaluation criterion alongside predictive accuracy. The metrics used to operationalize ecological cost are defined in Section 2.5, and the corresponding measurement infrastructure is described in Section 2.2.

In recent years, a paradigm has emerged in AI research that Schwartz et al. refer to as "Red AI" [1]. Red AI describes research approaches that primarily aim to improve prediction accuracy through the massive use of computational power, data, and parameters, while largely disregarding the associated infrastructural and energetic costs. The underlying dynamic driving this trend is that the relationship between model performance and model complexity is at best logarithmic [1]: a linear increase in accuracy requires an exponentially larger model or exponentially more training data. Empirically, since 2012 the compute used to train state-of-the-art models has been doubling approximately every 3.4 months [4]. This is particularly evident in large Transformer architectures. Strubell et al. document that training such models can require days of continuous computation across multiple GPUs, while foundation models such as GPT-3 scale to 175 billion parameters [2].

In response, the concept of *Green AI* has emerged as a counter-movement within the machine learning community [1]. Green AI is defined as research that yields novel results while explicitly accounting for computational costs, elevating efficiency to a primary evaluation criterion alongside predictive accuracy. Beyond the environmental dimension, the movement also addresses inclusivity: the prohibitive financial costs of training massive models increasingly restrict participation to a small number of well-resourced institutions [1]. A distinction must also be made between Green AI and *AI for Green*, where Green AI concerns reducing the ecological footprint of AI systems themselves, not applying AI to environmental problems [1], [5].

A core principle of the Green AI movement is evaluating environmental impacts across the entire lifecycle of an AI system, rather than focusing solely on the training phase [5], [6]. This holistic view encompasses data engineering, system integration, continuous retraining, and inference operations. Empirical work on data-centric Green AI illustrates the potential of this broader perspective: simple dataset preprocessing modifications, such as extracting smaller representative subsets or reducing feature counts, can decrease training energy consumption by up to 92 percent with negligible loss in accuracy [4].

To translate these principles into practice, researchers advocate for standardized reporting of computational work, energy consumption, and carbon emissions alongside accuracy results [1], [7]. Several tools have been developed to support this. The ML CO_2 Impact Calculator [3] and the `experiment-impact-tracker` [7] allow practitioners to estimate carbon emissions based on hardware configuration, execution time, and local grid carbon intensity. At the organizational level, the Green Software Foundation's Software Carbon Intensity (SCI) specification provides a standardized, consequential carbon accounting metric aimed at directly reducing AI carbon footprints rather than relying on market-based offsets [6], [8].

Realizing efficiency as a measurable criterion requires defining appropriate metrics. Floating-point operations (FLOPs) provide a hardware-independent estimate of computational work [1], but do not translate directly to energy or execution time, as these depend on hardware architecture, memory access patterns, and parallelization [7]. Training time is more intuitive but hardware-dependent and difficult to compare across environments. Energy consumption in kWh and CO_2 equivalents (CO_2eq) sidestep these limitations by capturing the actual physical and ecological footprint of a training run. Although tied to a specific hardware configuration and electricity grid, they provide directly comparable sustainability figures when experiments are conducted under controlled conditions [7], [8].

2.2 Energy and Carbon Emissions in Machine Learning

Operationalizing efficiency requires distinguishing two related but distinct quantities. Energy consumption is a hardware-level physical quantity, the product of power draw and time expressed in kilowatt-hours, where the actual power draw depends on hardware utilization and thermal design rather than being a fixed property of the components [7]. Carbon emissions then translate that energy into an ecological footprint, measured in carbon dioxide equivalents (CO_2eq), by multiplying it with the *carbon intensity* of the local electricity grid [3]. Two identical training runs can therefore produce the same energy consumption yet vastly different emissions, depending solely on the fuel mix of the grid supplying them. In practice, empirical studies have shown that identical workloads can vary in their carbon footprint by a factor of 40 depending on geographic location [3], and can even fluctuate based on the time of day the models are trained [8].

2.2.1 From Energy to Carbon Emissions

To understand the ecological footprint of machine learning, it is first necessary to distinguish between power and energy. Power, measured in Watts (W) or Joules per second, represents the instantaneous rate at which electricity is drawn by a system. Energy, on the other hand, is the total quantity of electricity consumed over a specific duration, calculated as the product of power and time ($E = P \cdot t$), and is typically expressed in kilowatt-hours (kWh) or Joules [7].

The execution of machine learning models relies on a combination of hardware components, primarily the Central Processing Unit (CPU), Graphics Processing Unit (GPU), and dynamic random-access memory (DRAM) [7]. Modern AI workloads rely heavily on GPUs because their highly parallelized architectures provide significant computational acceleration compared to traditional CPUs [8]. However, this increased throughput comes at a steep energetic cost: while CPUs typically consume between 10W and 150W under load, GPUs are significantly more power-hungry, often drawing between 250W and 350W [8]. As a result, the GPU typically accounts for the vast majority, often nearly three-quarters, of the total electrical energy consumed during deep learning training [8]. Nevertheless, the energy drawn by the CPU and DRAM remains a non-negligible factor that must be recorded for comprehensive and accurate sustainability accounting.

When evaluating the power profile of these computing components, the Thermal Design Power (TDP) is a frequently referenced specification. TDP defines the maximum heat flow generated by a CPU or GPU that its cooling system is designed to dissipate, serving as a theoretical indicator of the maximum power the component can draw [9]. Because direct, real-time power measurement can be complex, some simplified

methodologies attempt to estimate total energy consumption by simply multiplying a component's TDP by the total execution time of the model [9].

However, this approach is fundamentally flawed [7]. TDP represents an absolute peak thermal limit rather than an average operational consumption, and actual power draw fluctuates significantly based on hardware utilization, varying greatly between idle states and active load. Empirical studies demonstrate that estimating efficiency solely via TDP and time, while neglecting real-time hardware utilization and memory effects, leads to severe over- or underestimations of a model's true energy consumption and carbon footprint [7]. Consequently, reliable sustainability metrics require direct measurements of the hardware rather than abstract, TDP-based extrapolations.

Translating this measured energy consumption into a quantifiable environmental impact requires the concept of carbon intensity [3]. Carbon intensity represents the amount of greenhouse gas emissions produced per unit of electricity generated, expressed in grams of CO_2 equivalents per kilowatt-hour (gCO_2eq/kWh), and standardizes the global warming potential of various greenhouse gases into a single comparable value.

The operational carbon footprint of a machine learning model is derived by multiplying the total energy consumed (E, in kWh) by the carbon intensity of the local electricity grid (I), yielding $CO_2eq = E \cdot I$ [8]. Because different geographic regions rely on vastly different fuel mixes, from coal and natural gas to nuclear and renewables, carbon intensity varies drastically by location [7]. Empirical data illustrates this range clearly: training a model in Quebec, Canada, which is heavily reliant on hydroelectricity, faces a carbon intensity of approximately 20 gCO_2eq/kWh, whereas training the same model in a fossil-fuel-dependent region such as Iowa, USA, results in a carbon intensity of over 736 gCO_2eq/kWh [3].

Carbon intensity is not merely a static geographic property but also a temporally variable one [8]. The emissions factor of a grid can fluctuate significantly throughout the day based on the real-time availability of renewable sources, such as solar during daylight hours or wind at night. Consequently, accurately evaluating the ecological footprint of a machine learning model requires integrating both the spatial location and the temporal characteristics of the local energy grid into the final CO_2eq conversion.

2.2.2 Measurement Approaches

Accurately evaluating the energy consumption of machine learning models relies on a spectrum of methodologies, broadly categorized into physical measurements, estimation models, and on-chip sensors [9]. The baseline for ground-truth measurement is the External Power Meter (EPM), which tracks power directly at the wall outlet or motherboard. While EPMs capture the true consumption of the entire system, they are often costly to deploy at scale and cannot provide fine-grained decomposition to isolate the energy used by specific software processes or individual hardware components [9].

To achieve component-level granularity without external hardware, many approaches

utilize estimation models. These models calculate energy based on abstract hardware specifications, such as multiplying execution time by the Thermal Design Power (TDP) or counting Floating-Point Operations (FLOPs), or by leveraging Performance Monitoring Counters (PMCs) to track utilization rates [9]. However, as established in Section 2.2.1, relying solely on abstract analytical models or TDP often leads to severe inaccuracies under dynamic workloads, failing to capture the true, fluctuating real-time power draw of the hardware [7].

Consequently, modern sustainability accounting strongly prefers direct measurements obtained from vendor-specific on-chip sensors [7]. For GPUs, the industry standard is the NVIDIA Management Library (NVML), which polls instantaneous power consumption during execution. For CPUs and DRAM, the standard interface is the Running Average Power Limit (RAPL), provided by Intel and AMD, which accesses hardware performance counters to deliver accurate, fine-grained energy measurements directly from the CPU package [9].

Despite its high accuracy, a significant practical limitation of RAPL is its strict dependency on the underlying operating system [7]. RAPL interfaces are primarily supported on Linux environments and often require elevated administrator privileges to access the hardware counters securely. When direct sensor access is unavailable due to these constraints, software tracking tools often revert to problematic TDP-based estimation models as a fallback, which severely compromises measurement accuracy [9].

2.2.3 CodeCarbon

To practically implement the sustainability accounting principles advocated by the Green AI movement, researchers require standardized tools that bridge the gap between physical hardware metrics and environmental impacts. Initially, the machine learning community relied on a fragmented landscape of independent measurement approaches and preliminary software packages [10]. Early solutions largely consisted of web-based calculators, such as the ML CO_2 Impact calculator [3] and Green Algorithms [11], which required practitioners to manually input hardware specifications, execution times, and locations to estimate carbon footprints.

While these online calculators successfully raised awareness, they lacked real-time monitoring capabilities and had to rely on broad estimations rather than ground-truth physical power draw [10]. To address this, several independent Python packages were developed to track energy dynamically during model training by directly polling hardware sensors during execution, including the `experiment-impact-tracker` [7] and `CarbonTracker` [12]. However, the proliferation of tools with varying methodologies, carbon intensity data sources, and hardware support led to inconsistent reporting and discrepancies in carbon footprint measurements across the literature [10].

To unify these fragmented efforts into a reliable standard, the `CodeCarbon` initiative was established [10]. `CodeCarbon` is an open-source software library that evolved directly

from previous tracking projects, serving as the official successor to the Energy Usage tool [9] and integrating the efforts of the ML CO_2 Impact project [10]. By providing a unified tracking interface, it enables practitioners to quantify the environmental cost of their algorithms and report these metrics transparently [13].

To evaluate a model's energy consumption, the framework leverages the direct measurement approaches introduced in Section 2.2.2. Specifically, it interfaces with vendor-specific on-chip sensors, such as NVML for GPUs and RAPL for Intel and AMD processors, to collect fine-grained power draw measurements directly from the underlying hardware [9]. By periodically polling these hardware counters during execution, the tool computes the total electrical energy consumed by the computing components in kilowatt-hours.

Once the physical energy consumption is recorded, the framework translates it into greenhouse gas emissions using the conversion detailed in Section 2.2. The tool determines the carbon intensity of the local electricity grid based on the geographic location of the computing infrastructure, then multiplies the measured energy by this value to produce the final operational carbon footprint expressed in kilograms of CO_2-equivalents ($kgCO_2eq$) [8]. By combining hardware-level energy tracking with grid-level carbon intensity data in a single standardized framework, `CodeCarbon` automates the translation from computational work to ecological impact [7].

2.3 Supervised Classification Algorithms

In machine learning, supervised classification is the task of inferring a predictive model from historical, annotated data in order to make accurate categorical predictions on new data [14]. Formally, given a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ consisting of n examples, where each $x_i \in \mathcal{X}$ represents a vector of input features and $y_i \in \mathcal{Y}$ represents the corresponding ground-truth label, the objective is to learn a mapping function $f : \mathcal{X} \to \mathcal{Y}$ that reliably predicts the target output for any given input [15]. This reliance on an explicitly provided label space distinguishes supervised learning from unsupervised learning, where algorithms discover hidden structures in unannotated data, and from reinforcement learning, where an agent learns to make sequential decisions by maximizing a reward signal within an interactive environment.

The fundamental objective when learning f is achieving strong *generalization*: the model must perform reliably on new, unseen data drawn from the same underlying distribution rather than merely memorizing training inputs [16]. A central challenge during training is therefore finding the optimal balance to prevent *underfitting*, where the model is too simple to capture the true data patterns, and *overfitting*. Overfitting occurs when a model excessively adapts to the specific "sampling noise" present in the training data, fitting it so closely that its predictive performance on new, unseen data actively degrades [16], [17].

The structure of the label space $\mathcal{Y}$ determines the nature of the classification task. In binary classification, $\mathcal{Y} = \{0, 1\}$. When more than two categories are involved it becomes a multiclass problem with $\mathcal{Y} = \{1, \ldots, C\}$ [15]. Multiclass problems are typically handled via One-vs-Rest strategies or by extending binary formulations with a Softmax output that produces a normalized probability distribution across all C classes.

To optimize f during training, a loss function $\mathcal{L}(y, f(x))$ quantifies the discrepancy between predicted outputs and true labels [17]. For classification, the standard objective is Cross-Entropy Loss. In the binary case this reduces to the negative binomial log-likelihood, while for multiclass problems it generalizes to Categorical Cross-Entropy:

$$\mathcal{L} = -\sum_{k=1}^{C} y_k \log\big(p_k(x)\big) \tag{2.1}$$

where y_k is the binary indicator for the true class k and $p_k(x)$ is the model's predicted probability for that class. Cross-Entropy is preferred over Mean Squared Error for classification because it produces steeper, more informative gradients when a confident prediction is wrong, and provides a direct probabilistic interpretation of the model output. This foundational loss objective drives the gradient-based optimization for both the deep learning and advanced ensemble architectures discussed in the following subsections.

2.3.1 Logistic Regression

Logistic regression, formalized as a statistical classification method by Cox [18], is a parametric linear model that remains a standard baseline for binary classification tasks where interpretability and computational efficiency are paramount.

For a binary classification task ($\mathcal{Y} = \{0, 1\}$), logistic regression predicts the probability that an instance x belongs to the positive class. To ensure the output is bounded strictly between 0 and 1, the linear combination of input features and learned weights w with bias b is passed through the logistic sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, giving:

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}} \tag{2.2}$$

This functional form stems from modeling the log-odds (logit) as a linear function of the inputs: $\ln\left(\frac{P(y=1|x)}{P(y=0|x)}\right) = w^\top x + b$ [14].

The optimal parameters are estimated via Maximum Likelihood Estimation (MLE), which in practice is equivalent to minimizing the binary Cross-Entropy Loss over the training set [14]. Since no closed-form solution exists, iterative numerical solvers are required [14]. Standard approaches include second-order methods such as Newton-Conjugate Gradient (Newton-CG) and Limited-memory BFGS (L-BFGS), as well as first-order methods such as Stochastic Average Gradient (SAG) descent [19].

Logistic regression extends naturally to multiclass problems ($\mathcal{Y} = \{1, \ldots, C\}$) via a One-vs-Rest (OvR) strategy or by replacing the sigmoid with a Softmax function, yielding Multinomial Logistic Regression [14].

A key advantage of logistic regression is its interpretability: each feature weight directly represents its directional influence on the log-odds of the prediction [14]. It also trains quickly and is robust to overfitting on simple datasets [14]. Its fundamental limitation is linearity: without manual feature engineering, the model cannot capture non-linear patterns or feature interactions, making it considerably less expressive than the tree-based and deep learning architectures described in the following subsections [14].

2.3.2 Decision Trees

Decision trees represent a fundamental family of non-parametric machine learning algorithms that approach classification through a recursive binary partitioning strategy [20]. In this architecture, learning proceeds top-down from a root node through internal nodes that apply logical tests on input attributes, ultimately routing each sample to a terminal leaf node that assigns a discrete class label. Mathematically, this process partitions the multidimensional input feature space into a set of mutually exclusive, disjoint regions, where the model predicts a piecewise constant output value within each region [17].

The induction of a decision tree relies on a greedy split-finding algorithm. At each internal node, the algorithm evaluates candidate split points across all available features to find the partition that minimizes impurity in the resulting child nodes [17]. Two standard impurity criteria are used in practice: the Gini impurity, $G = \sum_{k=1}^{C} \hat{p}_k(1 - \hat{p}_k)$, which measures the probability of misclassifying a randomly drawn sample, and entropy-based information gain, $H = -\sum_{k=1}^{C} \hat{p}_k \log \hat{p}_k$, which quantifies the reduction in uncertainty achieved by a split [17], [20]. Gini impurity is favored by the CART algorithm and is the default criterion in `scikit-learn` [19].

The structural design of decision trees imparts specific inductive biases that make them particularly well-suited for tabular data, as documented by Grinsztajn et al. [21]. Because they naturally learn piecewise constant functions, decision trees can fit highly irregular, non-smooth patterns without requiring complex mathematical transformations. In contrast, neural networks exhibit an inductive bias toward smooth, continuous functions, which hinders their performance on such discontinuous tabular structures. Furthermore, decision trees operate strictly via axis-aligned splits and are therefore not rotationally invariant, which preserves the original orientation and independent meaning of heterogeneous tabular features [21], [22]. This reliance on the original feature basis also confers robustness against the uninformative features that frequently populate tabular datasets.

Despite these representational strengths, single decision trees suffer from signifi-

cant theoretical limitations. When grown deeply, a tree becomes highly susceptible to memorizing sampling noise, leading to high variance and severe overfitting [17]. Structural constraints such as a maximum depth or minimum number of samples per leaf partially mitigate this, but at the cost of increased bias and the need for careful tuning. To more systematically address these vulnerabilities while preserving the tree's ability to model irregular tabular data, the machine learning community shifted toward combining multiple tree predictors into ensemble methods [22]. This transition from single, unstable base learners to aggregated models forms the foundational motivation for the Random Forest and Gradient Boosting architectures described in the following sections.

2.3.3 Random Forest

As established in the previous section, the inherent instability, high variance, and overfitting risk of single decision trees serve as the primary motivation for aggregating multiple individual trees into powerful ensemble methods [22]. Random Forests, formally introduced by Leo Breiman, overcome these limitations by combining a large collection of tree predictors, where each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest.

The algorithm constructs this ensemble using a two-fold randomization strategy. First, it employs a technique known as *bagging* (bootstrap aggregating), originally proposed by Breiman [23], where each tree is grown on a new training set drawn at random with replacement from the original data. Random Forests extend this by additionally randomizing feature selection during the tree-growing phase: instead of evaluating all available features to determine the optimal split at each internal node, the algorithm selects the best split from a randomly chosen, restricted subset of features [22]. The constituent trees are typically grown to their maximum depth and are not pruned. To classify a new input vector, each tree in the ensemble casts a unit vote, and the forest outputs the majority class.

Breiman theoretically proved that the predictive accuracy of a Random Forest is governed by the interplay of two key parameters: the *strength* of the individual tree classifiers and the *correlation* between them [22]. An optimal forest consists of individually strong trees that produce decorrelated predictions. By combining bagging with random feature selection, the algorithm deliberately decreases the correlation between individual trees, which substantially reduces the variance of the overall ensemble without substantially compromising the predictive strength of the base learners. Building on this analysis, Breiman further demonstrated via the Strong Law of Large Numbers that as the number of trees grows, the generalization error almost surely converges to a limiting value, guaranteeing that the ensemble is inherently robust against overfitting with respect to the number of trees. While this convergence argument in the original paper was somewhat informal, formal consistency for a close variant of Breiman's

algorithm under standard assumptions was later established by Scornet, Biau, and Vert [24].

A crucial computational advantage of this architecture, particularly relevant for evaluating the efficiency of machine learning algorithms, is its inherent parallelizability [25]. Because the individual trees are entirely independent of one another, the training process can be distributed independently across multiple processing cores, allowing Random Forests to scale efficiently on multi-core CPU architectures [22], [25]. Additionally, the bagging mechanism naturally provides an internal validation metric: since each tree is trained on roughly two-thirds of the data, the remaining *out-of-bag* (OOB) samples can be used to estimate the generalization error without requiring a separate validation split [22].

Despite this parallelism, Random Forests carry a significant computational burden on large datasets. Each tree must be grown independently on a bootstrap sample of the full training set, and because the trees are deep and unpruned, the cost per tree grows with both the number of samples and the number of features evaluated at each split [25]. This cost is then multiplied across all M trees in the ensemble. As the dataset size grows into the millions of samples, the aggregate training cost becomes substantial, limiting the practical applicability of Random Forests on very large datasets without subsampling or other approximations.

2.3.4 Gradient Boosting and XGBoost

While Random Forests build independent, deep, unpruned trees in parallel, Gradient Boosting operates sequentially, constructing an ensemble of weak learners, specifically shallow trees, to progressively correct the errors of the overall model [17]. Introduced by Friedman, the gradient boosting machine frames this sequential training as a form of gradient descent in function space. Instead of adjusting numerical parameters, the algorithm iteratively adds a new decision tree to the ensemble that is fitted to the negative gradient of the loss function with respect to the predictions of the previous iteration. This allows the model to minimize arbitrary differentiable loss functions, making it a highly flexible and powerful approach for both regression and classification tasks.

XGBoost (Extreme Gradient Boosting), introduced by Chen and Guestrin, significantly extends this foundational concept to improve both predictive accuracy and computational scalability [26]. To combat the overfitting risks inherent in boosted ensembles, XGBoost modifies the traditional loss function by incorporating an explicit regularization term: $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$, where T is the number of leaf nodes and w represents the leaf weights [26]. Furthermore, while standard gradient boosting relies only on first-order gradients, XGBoost utilizes a second-order Taylor expansion of the loss function [26]. By calculating both the gradient and the Hessian (second derivative) for each training instance, the algorithm can optimize the tree structure more directly

and rapidly converge on the optimal split points. XGBoost also implements *shrinkage* and *column subsampling* to further prevent overfitting [26]. Shrinkage scales each new tree's contribution by a learning rate to slow convergence. Column subsampling mirrors a Random Forest's feature randomization but at the tree level. Finally, to natively support sparse data, XGBoost features a sparsity-aware split finding algorithm that automatically learns default directions for missing or zero values.

Despite these algorithmic optimizations, the exact greedy algorithm for finding the optimal tree splits requires sorting continuous feature values, which poses a significant computational bottleneck on massive datasets [26]. To accelerate this, XGBoost was extended to natively leverage Graphics Processing Units (GPUs) [27]. Formalized by Mitchell and Frank, the GPU-accelerated tree construction algorithm exploits the massive parallelism and high memory bandwidth of modern GPU architectures. Instead of relying on exact sorting, the *histogram method* bins continuous feature values into discrete buckets and aggregates gradient statistics per bin. By evaluating split points using highly optimized parallel primitives over these histograms, the entire decision tree can be constructed directly in device memory [27]. This histogram-based split-finding approach allows XGBoost to scale efficiently to millions of instances, achieving significant speedups over multi-core CPU implementations while fully accommodating the algorithm's second-order mathematical formulations. In their evaluation, Mitchell and Frank demonstrated the memory efficiency of this approach by processing a large-scale dataset of 11 million training instances and 28 features entirely within the memory of a single GPU, confirming its practical viability for large-scale classification tasks [27].

2.3.5 Multilayer Perceptron

The Multilayer Perceptron (MLP) is a feedforward artificial neural network architecture composed of sequential layers of neurons that apply linear transformations followed by non-linear activation functions. The Universal Approximation Theorem provides the mathematical justification for this architecture, guaranteeing that a standard feedforward network with as few as one hidden layer can approximate any Borel measurable function to an arbitrary degree of accuracy, provided it has a sufficient number of hidden units [28]. However, unlocking this immense representational capacity requires scaling the network's depth and width, which inherently increases both the model's susceptibility to overfitting and the computational burden of training.

Unlike decision trees, MLPs are optimized continuously using *backpropagation* [29] and stochastic gradient descent (SGD) or its variants to minimize a predefined loss function. To accelerate this optimization process in modern applications, the Adam optimizer is frequently deployed. Adam, which stands for Adaptive Moment Estimation, is a computationally efficient, first-order gradient-based optimization algorithm [30]. It computes individual adaptive learning rates for different parameters from estimates of the first and second moments of the gradients. By maintaining exponential moving

averages of the gradient and the squared gradient, and applying an initialization bias correction to counteract initial estimates being biased toward zero, Adam effectively adapts to the geometry of the objective function.

Historically, training deep architectures was severely hindered by the *vanishing gradient problem*. When utilizing traditional saturating non-linearities like the logistic sigmoid or hyperbolic tangent, the gradients of saturated neurons rapidly approach zero during backpropagation, making it nearly impossible for earlier layers to update effectively [31]. The widespread adoption of the Rectified Linear Unit (ReLU) largely resolved this bottleneck. By computing $y = \max(0, x)$, ReLU prevents gradients from vanishing for positive inputs and yields highly sparse representations [32]. Furthermore, its gradient is exactly 1 for positive inputs and 0 otherwise, making it highly efficient for training deep networks.

As MLPs scale in depth, they introduce considerable internal instability during training. During gradient descent, the distribution of inputs to any given hidden layer constantly changes as the parameters of all preceding layers are updated, a phenomenon termed *internal covariate shift* [33]. This forces the optimizer to use excessively low learning rates, substantially slowing down convergence. Batch Normalization overcomes this by explicitly standardizing the intermediate activations across each mini-batch to maintain a stable mean of zero and variance of one, before applying a learned affine transformation with parameters γ and β [33]. By fixing these distributions, Batch Normalization allows for the use of substantially higher learning rates and makes backpropagation highly resilient to parameter scale, resulting in substantially faster training times.

To prevent these deep networks from merely memorizing the sampling noise of the training data, regularization is required. *Dropout* provides a computationally elegant solution by randomly deactivating a fraction of hidden units during each forward pass [16]. This prevents neurons from developing complex co-adaptations where they solely rely on specific other units to correct their mistakes. At test time, scaling the outgoing weights seamlessly approximates an equally weighted geometric mean of all these shared-weight subnetworks, significantly reducing generalization error without the prohibitive cost of training multiple large models [16].

Similarly, because an MLP will eventually begin to overfit if training continues without constraint, *Early Stopping* is employed to monitor a separate validation set and halt the training process once the validation error stops decreasing [34]. Beyond acting as an implicit regularizer, this technique serves as a critical efficiency mechanism, saving significant computational time by avoiding redundant training epochs.

2.3.6 Tree-based Models vs. Deep Learning on Tabular Data

While deep learning has enabled tremendous progress on text and image datasets, its superiority on tabular data remains unproven [21]. For problems with data stored

in tabular formats consisting of heterogeneous features, ensembles of decision trees, particularly Gradient Boosted Decision Trees (GBDT) and Random Forests, are consistently recognized as the leading tools for both practitioners and machine learning competitions [15]. Extensive recent benchmarking has confirmed that tree-based models remain the superior choice on medium-sized tabular datasets, frequently outperforming deep learning architectures even before accounting for their superior training speed. However, this performance gap narrows significantly as the dataset size increases. Tree ensembles dominate in low-data regimes, but deep learning models become increasingly competitive when the number of training instances scales into the millions [21].

The performance gap between neural networks and tree-based models stems from their fundamentally different inductive biases and data requirements [21]. Neural networks are intrinsically biased toward overly smooth, low-frequency solutions. Consequently, they struggle to learn the highly irregular and non-smooth target functions that frequently characterize tabular datasets. In contrast, models based on decision trees learn piecewise constant functions and do not exhibit this smoothing bias, making them naturally adept at fitting these irregular patterns [21]. Beyond these structural differences, neural networks also exhibit poor sample efficiency compared to tree-based ensembles: deep learning models generally require substantially more data to learn useful representations without overfitting, making them ill-suited for very small tabular datasets where algorithms like Random Forests can quickly capture the underlying structure [21].

Furthermore, tabular datasets typically contain a large proportion of uninformative features [21]. Standard deep learning architectures, such as Multilayer Perceptrons (MLPs), are highly susceptible to these uninformative features, whereas tree-based models remain robust even when up to half of the dataset's features carry no information. Another critical distinction is rotation invariance. Neural networks are typically rotationally invariant, which degrades performance on tabular data because individual tabular features generally carry specific, distinct meanings. Applying rotational transformations mixes features with very different statistical properties, destroying the natural orientation of the data [21]. Conversely, the axis-aligned splits of decision trees preserve this information by avoiding arbitrary linear combinations of distinct features [21].

Despite these challenges, there has been significant research into designing deep learning architectures specifically tailored for tabular data [15]. Examples include differentiable tree ensembles and adaptations of the Transformer architecture, such as FT-Transformer, which applies self-attention mechanisms directly to feature embeddings. While these advanced models close the performance gap and achieve state-of-the-art results among deep networks, they still do not universally outperform GBDT implementations like XGBoost across all tasks, especially when hyperparameters are properly tuned [15]. Crucially, even when complex neural networks manage to match or

slightly exceed the predictive accuracy of tree-based ensembles on datasets of millions of instances, they demand substantially more computational resources and training time to achieve these results [21]. This fundamental trade-off between predictive gain and computational cost reinforces the need to evaluate models not solely on accuracy, but also on their overall resource and energy efficiency.

2.3.7 Algorithmic Suitability for CPU vs. GPU

In the context of Green AI, optimizing energy efficiency requires aligning the computational profile of a machine learning algorithm with the architectural strengths of the underlying hardware [1]. While it is common practice to universally accelerate training using GPUs, this approach is not intrinsically energy-efficient for all algorithms. The fundamental divide in hardware suitability lies in the distinction between dense, regular parallelism and irregular, sequential branching.

Hardware platforms are designed with opposing optimization goals: GPUs are optimized for aggregate throughput on parallel tasks, whereas CPUs are optimized for low latency on sequential, single-threaded tasks [35]. GPUs achieve this through a *Single Instruction, Multiple Threads* (SIMT) architecture that executes thousands of lightweight threads concurrently to hide memory latency. This design heavily favors dense, regular computational tasks with uniform memory access patterns, most notably the large-scale matrix multiplications that form the computational core of neural network training. When operations can be uniformly distributed across thousands of cores simultaneously, the GPU achieves maximum utilization.

Conversely, CPUs rely on large caches, branch prediction, and speculative execution to handle irregular control flows and sequential decision logic with minimal latency. Algorithms that depend on dynamic branching, non-contiguous memory access, or sequential data traversal perform poorly on GPUs: in a SIMT environment, divergent threads within a warp must serialize, negating the throughput advantage and leaving cores idle [27]. For these workloads, the CPU is the more efficient hardware choice.

Despite the higher peak power draw of GPUs, their throughput on suitable workloads means they process substantially more training samples per joule than a CPU running the same task. A GPU can therefore deliver superior energy efficiency for well-matched workloads despite consuming more power in absolute terms. This advantage is not universal: at small dataset scales, the GPU's higher idle and transfer overhead can make CPU training the more energy-efficient option overall.

2.4 Dataset Scaling and Learning Curves

In predictive modeling, the relationship between the volume of available training data and the resulting classification performance is formally modeled using a learning curve

[36]. Empirical learning curves consistently demonstrate that performance improves as more data becomes available, but the rate of improvement eventually diminishes, typically following an inverse power law $P(n) \propto n^{\alpha}$ with $\alpha \in (0, 1)$ [14], [36]. This creates a fundamental tension in the context of Green AI: while computational and energy costs scale roughly linearly with dataset size, performance yields only sub-linear growth, making it increasingly ecologically expensive to achieve stronger results simply by adding more data [1]. Achieving a marginal accuracy gain at the upper end of a learning curve demands a disproportionately large investment in additional data, time, and energy.

Conversely, when dataset sizes are heavily restricted, classifiers generally experience degraded performance because smaller datasets contain fewer representative examples, leaving the model unable to generalize underlying patterns [37]. The risk of overfitting also increases: with limited samples, a model may memorize training noise rather than learn true structure. Crucially, however, the overall performance of a classifier depends more heavily on how well the dataset represents the underlying data distribution than on its sheer size [37]. A small but highly representative dataset can yield better generalization than a large but biased one.

Different classes of machine learning algorithms exhibit different sensitivities to dataset scale. Deep learning architectures are highly data-dependent: they must fit a large number of parameters via backpropagation, meaning more data directly enables better adjustment and generalization [37]. Tree-based models are also sensitive at the smaller end of the scale, as recursive partitioning means predictions at deeper nodes rest on increasingly small subsets of examples, making tree depth unstable when the initial dataset is limited [22], [37]. In contrast, simpler linear models serve as robust baselines: their constrained functional form imparts a higher model bias but a much lower variance, making them far less prone to overfitting on limited datasets [14].

A complex model's absolute accuracy must therefore be weighed against its computational footprint and the diminishing returns of the learning curve. This trade-off is especially pronounced in a Green AI context, where the ecological cost of scaling up model complexity or dataset size may exceed the marginal accuracy gain.

2.5 Evaluation Metrics

Assessing the ecological efficiency of a machine learning algorithm requires more than measuring its predictive performance in isolation. An algorithm that achieves high accuracy at disproportionate computational cost is not unconditionally preferable to one that is slightly less accurate but significantly more resource-efficient [1]. This section therefore introduces the theoretical basis for three families of metrics. The first, classification quality, measures how accurately a model predicts class labels. The second, resource consumption, quantifies the energy expenditure and associated carbon

emissions of training. The third, composite efficiency, integrates both dimensions into a single score that captures the trade-off between predictive performance and ecological cost. The specific metrics and their experimental implementation are described in Section 4.4.

At the core of evaluating classification quality is the *confusion matrix*, which cross-tabulates a model's predictions against the true labels. For a binary classification problem, this matrix consists of four fundamental components: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). This concept extends naturally to multiclass problems by tracking predictions across a $C \times C$ grid, forming the basis for deriving all subsequent performance metrics [14].

Table 2.1: Binary confusion matrix with the four fundamental prediction outcomes.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

The most intuitive of these metrics is accuracy, defined as the overall proportion of correct predictions: Accuracy $= \frac{TP+TN}{N}$. While simple to interpret, accuracy exhibits a critical vulnerability when applied to datasets characterized by an uneven distribution of labels, known as *class imbalance*. For example, in a dataset where 99% of the instances belong to a single majority class, a trivial classifier that ignores all input features and simply predicts the majority class every time will achieve 99% accuracy. Despite this high score, the model learns no underlying patterns and fails completely at identifying the minority class.

This limitation motivates the use of more robust metrics, specifically precision and recall. Precision ($\frac{TP}{TP+FP}$) measures the proportion of positive predictions that are actually correct, while recall ($\frac{TP}{TP+FN}$) measures the proportion of actual positive instances successfully identified. Unlike accuracy, precision and recall evaluate performance entirely independently of true negatives, making them highly resilient to datasets where a large, heterogeneous majority class dominates the negative counts [38]. Because improving precision often comes at the expense of recall and vice versa, the F1-score combines them into a single metric calculated as their harmonic mean:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.3}$$

The harmonic mean is explicitly chosen over the arithmetic mean because it heavily penalizes extreme values. A model that achieves near-perfect recall but abysmal precision will yield a low F1-score, forcing the classifier to balance both dimensions effectively. Consequently, the F1-score provides a more encompassing summary of overall model performance and overcomes the representation problems caused by imbalanced data distributions.

When dealing with multiclass or imbalanced datasets, the F1-score can be aggregated in several ways. Macro F1 calculates the metric independently for each class and averages them, treating all classes equally regardless of their size [38]. Micro F1 aggregates the global contributions of TP, FP, and FN across all classes before calculating the score, an approach that inherently favors larger classes [38]. Weighted F1 calculates the score for each class independently but averages them using the true support of each class as a weight.

Among these aggregation strategies, Weighted F1 is particularly well-suited to imbalanced settings. By weighting each class's score according to its true support, it yields a more representative picture of overall model performance than either Macro F1, which gives disproportionate influence to rare classes, or simple accuracy, which is entirely dominated by the majority class.

Measuring classification quality, however, captures only one dimension of model evaluation. Evaluating models under the Green AI paradigm requires moving beyond pure predictive quality to measure the physical and environmental costs of achieving that performance [1]. The primary resource metrics tracked include training time (measured in seconds), energy consumption (measured in kilowatt-hours, kWh), operational carbon emissions (measured in kg of CO_2eq), and inference latency (seconds per sample). The foundational definitions and physical measurement approaches for these quantities are established in Section 2.2.

To holistically evaluate machine learning models, researchers increasingly employ composite efficiency metrics that weigh predictive performance against these resource expenditures in a single score. The general methodological motivation behind these composite metrics is to provide a unified heuristic for identifying algorithms that offer the optimal trade-off between classification quality and ecological impact [1]. In the literature, such approaches include the Energy-Efficiency Ratio, Performance-per-Watt (which tracks operations or inferences per Joule) [39], and FLOPs-per-Sample (or per-inference) [1].

However, composite metrics possess inherent theoretical weaknesses. First, the resource-based components of these metrics, such as energy consumed or execution time, are intrinsically hardware-dependent. A composite score calculated in one environment is tied to the specific computational architecture and local energy grid, meaning it cannot be universally generalized or directly compared across different settings [1], [8]. Second, the weighting between performance and resource cost is highly scenario-dependent. Determining how much a marginal gain in predictive accuracy is worth compared to the energy required to achieve it remains an open question that depends heavily on the specific priorities or economic utility of the deployment scenario [7].

2.6 Cross-Validation

The simplest approach to evaluating a model's generalization capability is the *hold-out method*, where the available data is partitioned into a single training and test split (for example, an 80/20 ratio). While computationally efficient, this approach is highly sensitive to the specific data partition and exhibits high variance in its performance estimate, particularly when the dataset is small [14]. To mitigate this instability, K-fold cross-validation is widely employed as a more rigorous evaluation technique [14].

The algorithm partitions the dataset into K roughly equal-sized folds. The model is trained K times. In each iteration, $K-1$ folds are used to fit the model, while the single held-out fold is used for testing [14]. Formally, the cross-validation estimator is defined as:

$$\mathrm{CV}(K) = \frac{1}{K}\sum_{k=1}^{K} L\big(y_k,\ \hat{f}^{-k}(x_k)\big) \tag{2.4}$$

where L is the chosen evaluation metric, y_k are the true labels of fold k, and $\hat{f}^{-k}$ denotes the model trained on all folds except the k-th [14]. The standard deviation of L across all K folds provides a quantitative measure of the model's stability across different data samples.

Selecting the number of folds K introduces a fundamental bias-variance trade-off. A small value of K is computationally less expensive but yields a higher estimation bias, as the model is trained on a significantly smaller fraction of the original data [14]. Conversely, setting $K = N$, known as *Leave-One-Out Cross-Validation* (LOOCV), trains the model N times using $N-1$ samples for each iteration. While LOOCV produces a nearly unbiased estimate of the test error, it is computationally prohibitive for large datasets and can exhibit high variance. Because the N training sets differ by only a single sample, the resulting estimators are highly correlated. Unlike independent estimators, whose averaged variance shrinks by a factor of $\frac{1}{N}$, the variance of correlated estimates does not diminish at this rate, so the averaging step of LOOCV provides little variance reduction [14]. To balance these competing factors, $K = 5$ and $K = 10$ are established as the standard recommendations in statistical learning, offering an optimal compromise between computational feasibility, low bias, and acceptable variance [14]. The specific justification for $K = 5$ is detailed in Section 4.5.

Finally, when evaluating datasets characterized by class imbalance, standard random partitioning may result in folds that do not accurately represent the underlying target distribution. To address this, stratified cross-validation is applied, which ensures that the original class ratio is preserved within every fold. This is directly consistent with the choice of Weighted F1 as the primary evaluation metric: the same class imbalance that motivates a support-weighted metric must also be faithfully reproduced in each fold's test set, ensuring the evaluation procedure and the metric it produces remain coherent.

2.7 Hyperparameter Optimization

Machine learning models are governed by two distinct categories of configurable quantities. Parameters, such as the weights of a neural network or the split thresholds of a decision tree, are internal to the model and are learned directly from training data by minimizing a loss function [40]. Hyperparameters, by contrast, are external configuration choices that must be fixed prior to training: examples include the learning rate of an optimizer, the maximum depth of a tree, or the number of estimators in an ensemble [40]. Because hyperparameters govern the model's capacity, regularization strength, and optimization dynamics, their values have a direct and often decisive impact on generalization performance [40], [41].

The goal of hyperparameter optimization (HPO) is to identify the configuration from a predefined search space Λ that minimizes the expected generalization error, where Λ is the Cartesian product of the individual value ranges of each hyperparameter and the error is estimated via cross-validation [41]. As the number of hyperparameters grows, exhaustive evaluation of Λ becomes exponentially expensive [40], [41]. This combinatorial explosion motivates the development of more efficient search strategies, which are described in the following subsections.

2.7.1 Grid Search and Random Search

The most traditional approach to hyperparameter tuning is Grid Search, an exhaustive strategy that evaluates the Cartesian product of a user-specified, finite set of values for each parameter [40]. While conceptually simple, Grid Search suffers severely from the *curse of dimensionality* [40], [41]. As the number of hyperparameters increases, the computational complexity grows exponentially. For instance, optimizing five hyperparameters with ten candidate values each across four models and three datasets would require an exhaustive evaluation of 150,000 distinct configurations. This combinatorial explosion renders Grid Search computationally prohibitive for complex machine learning architectures with limited budgets [40].

To overcome the inefficiency of exhaustive evaluation, Random Search was proposed as a highly effective alternative [41]. Instead of testing predetermined values on a rigid grid, Random Search selects a predefined number of parameter combinations randomly from the specified search space distributions [40]. Empirical and theoretical analyses demonstrate that Random Search is significantly more efficient than Grid Search in high-dimensional spaces because hyperparameter response surfaces typically exhibit a low effective dimensionality [41]. In most predictive tasks, only a small subset of hyperparameters substantially impacts the generalization error, while others remain largely irrelevant [41]. Because Grid Search forces allocations across all dimensions equally, it wastes the vast majority of its computational budget exploring irrelevant parameters. Random Search, by contrast, tests a unique value for every parameter

in every trial, allowing it to thoroughly explore the relevant subspaces with far fewer evaluations [41].

Despite their differences in candidate selection, both Grid Search and Random Search share a fundamental methodological limitation: they are entirely uninformed algorithms [40]. Every trial in their search process is independent, meaning neither method utilizes the performance results of previously evaluated configurations to influence the selection of future points [40]. Consequently, they inevitably waste massive amounts of time and computing resources evaluating poorly performing areas of the configuration space without learning from past failures [40]. This inherent inefficiency establishes the primary motivation for adopting sequential, history-dependent strategies such as Bayesian Optimization.

2.7.2 Bayesian Optimization

To overcome the limitations of uninformed search strategies, Bayesian Optimization (BO) offers a sequential, model-based approach that utilizes the history of previously evaluated configurations to intelligently determine the next hyperparameter values to test [40]. BO operates using two primary components: a probabilistic *surrogate model* that approximates the true, computationally expensive objective function, and an *acquisition function*, such as Expected Improvement (EI), which selects the optimal next point for evaluation [40], [42]. The acquisition function explicitly balances the fundamental optimization trade-off between *exploration* (sampling from under-explored regions of the parameter space where uncertainty is high) and *exploitation* (sampling from regions already known to yield promising results) [40].

While traditional BO relies on Gaussian Processes (GPs) as surrogate models to directly estimate the conditional probability $p(y|x)$ of the objective value y given the hyperparameters x, the Tree-structured Parzen Estimator (TPE) takes a fundamentally different approach [42]. Instead of modeling $p(y|x)$ directly, TPE models the parameter distribution $p(x|y)$ and the marginal $p(y)$ [42]. It achieves this by creating two separate generative densities: one for hyperparameter configurations that perform below a specific threshold (the best-performing regions), and another for configurations that perform above it [40], [42]. The expected improvement is then maximized by drawing candidates that have high probability under the good density and low probability under the poor density [42].

This inverse modeling formulation grants TPE a significant advantage in complex configuration spaces. Because the Parzen estimators are organized in a tree structure, TPE naturally retains conditional dependencies between variables [40]. This allows it to efficiently navigate mixed search spaces containing continuous, discrete, categorical, and conditional hyperparameters, a scenario where standard GP models inherently struggle [40], [42].

Furthermore, TPE offers a substantial computational advantage over both GPs and

exhaustive search methods. While GP-based models scale cubically with the number of past observations $\mathcal{O}(n^3)$, making them prohibitively slow as the search history grows, TPE achieves a much lower time complexity of $\mathcal{O}(n \log n)$ [40]. By sequentially directing the search only toward the most promising regions and maintaining low computational overhead per trial, sequential strategies like TPE bypass the exponential combinatorial explosion that plagues Grid Search, enabling the efficient optimization of complex machine learning architectures with many hyperparameters [40].

Table 2.2: Comparison of HPO strategies (n: trials, d: hyperparameters).

Method	**Strategy**	**History**	**Complexity**	**Best for**
Grid Search	Exhaustive	No	$\mathcal{O}(n^d)$	Few hyperparameters
Random Search	Random sampling	No	$\mathcal{O}(n)$	Low effective dim.
Bayesian (GP)	Sequential, model-based	Yes	$\mathcal{O}(n^3)$	Moderate spaces
TPE (`Optuna`)	Sequential, model-based	Yes	$\mathcal{O}(n \log n)$	Complex/conditional

2.7.3 Optuna as a Practical Framework

`Optuna` is a modern hyperparameter optimization framework designed around a *define-by-run* API [43]. Unlike traditional tools that require the search space to be statically defined prior to execution, `Optuna` constructs the hyperparameter search space dynamically within the objective function during runtime, accommodating complex dependencies without relying on rigid external configurations. `Optuna` utilizes the Tree-structured Parzen Estimator (TPE) as its default sampling algorithm, allowing for efficient exploration of these dynamically generated spaces. To ensure reproducibility, the stochasticity of the TPE sampler can be controlled by fixing a global random seed.

Beyond efficient sampling, `Optuna` accelerates the optimization process through automated early stopping, formally referred to as *pruning*. A pruner periodically monitors the intermediate learning curves of iterative algorithms and preemptively terminates unpromising trials that fall below a predefined performance threshold, such as the median performance of previously completed trials. By allocating computational resources exclusively to the most promising configurations, pruning drastically reduces the overall runtime [43]. The specific details regarding whether and how pruning is applied are deferred to Section 5.4.

Ultimately, the adoption of an efficient framework like `Optuna` is not merely a mathematical convenience, but a fundamental Green AI strategy. The total computational and ecological cost of a machine learning result grows linearly with the number of hyperparameter trials executed [1]. By utilizing Bayesian Optimization to strategically minimize the number of required evaluations, and employing pruning to terminate poor configurations early, the overall energy consumption and resulting carbon footprint of the entire model development process are significantly reduced.

2.8 Chapter Summary

This chapter established the theoretical foundations for the empirical study. The Green AI framework motivated the treatment of ecological cost as a first-class evaluation criterion alongside predictive accuracy. The energy and emissions section formalized how power draw translates into carbon footprint through grid carbon intensity, established why TDP-based estimation is unreliable, and introduced `CodeCarbon` as the standardized measurement tool. The algorithms section described the five classifiers under examination, their inductive biases on tabular data, and the conditions under which GPU acceleration is genuinely energy-efficient rather than merely faster. Dataset scaling and learning curves established the sub-linear relationship between data volume and performance gains, reinforcing the Green AI argument that ecological costs can outpace accuracy returns. The evaluation metrics section introduced Weighted F1 as the primary classification quality metric and outlined the theoretical limitations of composite efficiency metrics. Cross-validation was formalized as the estimation procedure, with stratification linking directly to the metric choice under class imbalance. Finally, hyperparameter optimization was covered from exhaustive search through Bayesian Optimization to Optuna, with HPO trial count identified as an ecological cost in its own right. These concepts collectively provide the vocabulary and methodology for the experimental design presented in Chapter 4.

Chapter 3

Related Work

Chapter 4

Experimental Design

This chapter outlines the experimental design used to evaluate the ecological efficiency of different machine learning classification algorithms. It describes the datasets, models, evaluation metrics, hyperparameter tuning and measurement setup that form the basis of this empirical analysis.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO2 output scale with increasing neural network complexity (number of layers and neurons)?
- How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?
- How accurate are software-based energy estimation tools compared to direct hardware sensor measurements, and how does this affect reported CO_2 emissions?

Letzte Unterfrage in der Diskussion ausführlicher behandeln: CodeCarbon unterschätzt CPU-Verbrauch empirisch um Faktor 2–9× (gemessen via LibreHardwareMonitor CPU Package Power Sensor). Begründung: TDP-basierte Schätzung skaliert linear mit CPU-Auslastung, bildet aber weder Boost-Verhalten noch Uncore-Komponenten ab. Konkrete Messwerte als Beleg vorhanden (z.B. LogReg Credit: 6.6 W CodeCarbon vs. 57.8 W HardwareMonitor; LogReg Higgs: Faktor ∼2.3×).

RQ4 (Scaling): Kurz das experimentelle Design für den Subset-Run beschreiben. Die Kernidee: statt verschiedener Datasets wird *derselbe* Datensatz (HIGGS) in fünf Größen aufgeteilt (100k, 500k, 1M, 5M, 11M Instanzen via `TEST_NROWS` Umgebungsvariable). Alle Modelle außer Random Forest werden mit denselben Hyperparametern (aus dem HIGGS-Tuning) auf jeder Größe trainiert. So wird der Dataset-Größen-Effekt isoliert, ohne dass unterschiedliche Hyperparameter oder Feature-Räume den Vergleich verfälschen. Ergebnis: CO2, Trainingszeit und F1 als Funktion von nrows für alle Modelle.

4.1.1 Break-Even Analysis

To determine at which dataset size GPU training becomes more energy-efficient than CPU training for XGBoost, a break-even analysis was designed as part of this study. The analysis is based on nine HIGGS subsets ranging from 1,000 to 11,000,000 instances, using identical hyperparameters for both devices. This controlled setup isolates the effect of dataset size without confounding factors from different feature spaces or hyperparameter configurations.

The relationship between dataset size and CO_2 emissions is modelled as a power law, fitted separately for CPU and GPU by applying a linear regression to $\log_{10}(\text{nrows})$ versus $\log_{10}(CO_2)$. A log-log fit is used because emissions scale super-linearly with data size, and a linear model would systematically misrepresent the relationship. The break-even point n_{BE} is then derived analytically as the intersection of the two fitted lines:

$$\log_{10}(n_{BE}) = \frac{b_{\text{GPU}} - b_{\text{CPU}}}{a_{\text{CPU}} - a_{\text{GPU}}} \tag{4.1}$$

where a and b are the slope and intercept of each fit. Below n_{BE}, CPU training is more energy-efficient; above it, GPU training is. The results are presented in Chapter 6.

To answer these questions multiple classification algorithms and datasets were used in the experimental phase of this empirical analysis.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [9]. To evaluate how this relationship varies across different scales, three classification datasets were selected that span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and F1-score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	**Instances**	**Features**	**Classes**	**Scale**
Wine [44]	178	13	3	Small
Default of Credit Card Clients [45]	30,000	23	2	Medium
HIGGS [46]	11,000,000	28	2	Large

The *Wine* dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The *Default of Credit Card Clients* (hereafter: *Credit*) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The *HIGGS* dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [47]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, five classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Logistic Regression serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive accuracy, making it well

Table 4.2: Overview of classification models used in this study

Model	Type	Device	Role
Logistic Regression [18]	Linear	CPU	Baseline
Random Forest [22]	Ensemble	CPU	Mid-complexity
XGBoost [26]	Gradient Boosting	CPU & GPU	Mid-complexity
Multilayer Perceptron	Neural Network	GPU	High-complexity

suited to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest was selected as a mid-complexity representative of tree-based ensemble methods, as extensively described in Section 2.x . Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, XGBoost was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study. The suitability of GPU-accelerated XGBoost for datasets at this scale has been demonstrated by Mitchell and Frank, who show that the histogram-based GPU implementation can process a dataset of 10 million instances with 28 features entirely within GPU memory [27].

The MLP was selected as the most complex model, representing the deep learning category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. A more detailed description of the MLP architecture is provided in Section 2.x.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions, as further described in Section 2.5. Energy consumption in kWh and CO_2 emissions in kg were chosen as

the primary resource metrics, directly reflecting the ecological cost of model training. It should be noted that all experiments were conducted on a single hardware setup, which is described in detail in Section 5.1.1.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [39]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the *Efficiency-Weighted F1* (EWF1), defined as:

$$\text{EWF1} = \frac{F_1}{1 + \lambda_1 \cdot \hat{t} + \lambda_2 \cdot \hat{c}} \tag{4.2}$$

where $\hat{t}$ and $\hat{c}$ denote the min-max normalized training time and CO2 emissions per dataset respectively. The weights were set to $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, assigning greater importance to carbon emissions than training time, reflecting the ecological focus of this study. Higher scores indicate a more favorable trade-off between predictive performance and resource consumption.

It should be noted that both $\hat{t}$ and $\hat{c}$ are inherently hardware- and location-dependent, as training time varies with the underlying hardware configuration and CO2 emissions are tied to the carbon intensity of the local electricity grid [1]. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other hardware or energy infrastructures.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a cross-validation strategy was employed rather than a single train-test split, as described in Section 2.x.

Specifically, K-Fold cross-validation was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [14]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

To ensure reproducibility, a fixed random state was used for all cross-validation splits, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 5.1.2.

4.6 Hyperparameter Tuning

To ensure a fair comparison across all models, hyperparameter tuning was applied to each model prior to evaluation. Without tuning, default parameters tend to favor certain model families over others, potentially skewing the comparison in terms of both predictive performance and resource consumption [48].

Hyperparameter optimization was performed using Optuna [43], an optimization framework based on Bayesian search (see Section 2.x).

Hier in 02 die genaue Erklärung

Grid search was considered but deemed infeasible given the scale of experiments: assuming five hyperparameters per model with ten candidate values each, evaluated across three datasets and four models, a full grid search would require approximately 150,000 evaluations. Optuna was therefore configured to run 40 trials per model per dataset, striking a balance between tuning thoroughness and resource consumption, the latter being a relevant factor given that tuning runs contribute to the overall energy budget of this study, as described in Section 4.4.

Weighted F1-score was used as the optimization objective, as it accounts for class imbalance more robustly than accuracy, providing a more reliable estimate of model quality across datasets with varying class distributions.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. A globally tuned model would not represent optimal performance on any individual dataset and would therefore introduce bias into the comparison.

Chapter 5

Implementation and Measurement

This chapter describes how the experimental pipeline was built and how measurements were collected. It covers the hardware and software environment, the data loading architecture, each model implementation, the hyperparameter tuning setup, and the emissions measurement infrastructure including the custom CPU power monitoring solution.

5.1 Experimental Setup

5.1.1 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university's shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 graphics processing unit (12 GB GDDR6 memory, 5,888 CUDA cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz), and 32 GB of DDR4 RAM. The system runs Windows 11 with all code implemented in Python 3.13.0.

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP

Table 5.1: Hardware specification of the experimental system

Component	Specification
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
RAM	32 GB DDR4
Operating System	Windows 11

models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the local electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.1.2 Software Environment and Reproducibility

All experiments were implemented in Python 3.13.0. The main libraries used are described below.

Scikit-learn [19] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for cross-validation, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and `f1_score` and `make_scorer` for evaluation metrics.

The XGBoost library [26] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [49] was used as the deep learning backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [50] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna, as described in Section 4.6.

Carbon emissions and energy consumption were tracked using CodeCarbon [13], which is described in detail in Section 5.5.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, cross-validation folds, and model initialization. A `requirements.txt` file is provided

so that the exact library versions used in this study can be reproduced.

The implementation code was developed with the assistance of Claude Code [51], an AI-powered development tool by Anthropic. All generated code was reviewed, tested, and integrated by the author.

To ensure fully reproducible results across all experiments, a global constant `RANDOM_STATE` with a value of 42 is used in every model script. It is passed to the `KFold` splitter and all model constructors. For the MLP, the seed is additionally set via `torch.manual_seed(RANDOM_STATE)` to guarantee deterministic weight initialization.

The evaluation relies on `KFold` with five splits and shuffling enabled, executed through `cross_validate()`. Performance is measured using a scoring dictionary that tracks both accuracy and weighted F1 score via `make_scorer(f1_score, average="weighted")`. Final scores are the mean across all five folds.

5.2 Dataset Integration

The three datasets used in this study, *Wine*, *Credit*, and *HIGGS*, were introduced in Section 4.2. This section describes how they are integrated into the software architecture, covering the central configuration file, the data loading pipeline, and dataset-specific constraints.

To ensure a consistent foundation across all model scripts, a central configuration file (`config.py`) was created. It serves as a single source of truth for all dataset-related settings, so that each script accesses the same parameters without redundant definitions.

For each dataset, the configuration stores the file path, the name of the target column, and an optional row limit (`nrows`). The row limit was particularly relevant for the HIGGS dataset, which contains 11 million instances, allowing experiments to be run on smaller subsets where needed. Beyond these common fields, the configuration also captures dataset-specific properties. For example, the Credit dataset includes a header row that must be skipped during loading, and the Wine dataset does not contain column names in the source file, which are therefore defined in the configuration directly. Additionally, a label offset is applied to the Wine target column, as its class labels are encoded as 1, 2, and 3 rather than starting at 0, which is required by the classifiers used in this study.

The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here, ensuring that all scripts use identical values for random seeding and cross-validation, which is essential for the comparability of results.

To read the datasets from their respective files, a custom `load_data` method was implemented. Depending on the dataset, the appropriate loading mechanism is selected. The *Wine* and *Credit* datasets are loaded using pandas' `read_csv` function. For the *Higgs* dataset, `read_parquet` is utilized. This was a deliberate decision to save storage space by storing the large *Higgs* dataset as a Parquet file rather than a standard CSV.

Following the initial loading phase, the *Higgs* data is reduced to a representative subset using a sampling approach controlled by the `nrows` parameter. Once the data size is managed, any columns specified in the configuration file are dropped from the respective dataset. The data is then split into the feature matrix X and the target vector y. If an offset is defined in the config, which is currently only the case for the *Credit* dataset, it is applied as described in the previous section. Finally, the method returns X and y for further use in the subsequent scripts. This process is visualized in Figure 5.1.

Due to the substantial scale of the *Higgs* dataset, several constraints were implemented to ensure the stability of the execution environment. Training a `Random Forest` model on this specific data led to unmanageable processing times. For this reason, the `Random Forest` algorithm is explicitly bypassed for the *Higgs* dataset and the program terminates using `sys.exit(0)` to prevent system crashes. This allows the process to stay within the boundaries of the available hardware resources.

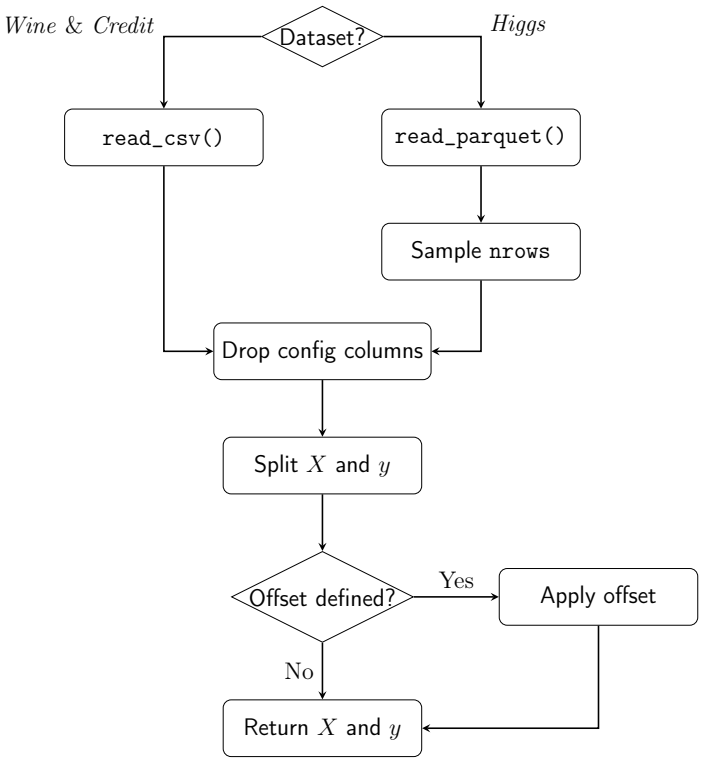

Figure 5.1: Visualization of the data loading pipeline and preprocessing steps within the `load_data()` method.

5.3 Model Implementations

As outlined in Section 4.3, the following classification algorithms were employed to address the research questions mentioned previously. This section therefore details the specific implementation of each model.

5.3.1 Logistic Regression

The first baseline model is a standard logistic regression. To ensure the model processes all features on an equal scale, the algorithm is embedded in a pipeline starting with a `StandardScaler`. This initial scaling step is crucial for the stability and overall performance of the subsequent classification.

The regression is configured to use the Stochastic Average Gradient (SAG) algorithm by setting the `solver` parameter to `sag`. This specific solver is highly optimized for large datasets. It significantly reduces the memory footprint and required training time, which is especially important when evaluating the extensive *Higgs* dataset. To ensure proper convergence, the maximum number of iterations is capped at 1000 using the `max_iter` parameter.

This model setup deliberately excludes any hyperparameter optimization. Instead, it acts as a stable baseline to evaluate the raw computational time without the extra overhead of complex search processes.

It is worth noting that the SAG solver is single-threaded and runs on a single CPU core, resulting in roughly 10% utilisation on the 8-core system used in this study. The `cross_validate()` call also uses no `n_jobs` parameter, so the five folds run sequentially. Each model in this study runs with its natural parallelisation behaviour; the implications of this for energy comparability are discussed in Section 5.5.2.

5.3.2 Random Forest

The second baseline algorithm is the Random Forest classifier. This method belongs to the family of ensemble learning techniques. Unlike logistic regression, this algorithm does not require prior feature scaling. To maximize computational efficiency, the execution is fully parallelized across all available CPU cores by setting the `n_jobs` parameter to `-1`. This configuration allows the processor to run at maximum capacity, which directly influences the subsequent energy consumption tracking and overall runtime measurements.

As established in Section 5.2, Random Forest is explicitly excluded from the *Higgs* dataset due to unmanageable training times at 11M instances. Including it would produce extreme runtimes without contributing meaningful insight into the scaling behaviour studied here.

To ensure maximum predictive capabilities for the remaining data, the model configurations were optimized using Optuna. The optimization process specifically targeted maximizing the weighted F1 score. For strict reproducibility, the final parameter combinations for the *Wine* and *Credit* datasets are documented in Table 5.2.

hier noch die korrekten werte nach dem finalen run eintragen

Table 5.2: Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.

Parameter	*Wine*	*Credit*
`n_estimators`	221	381
`max_depth`	5	9
`min_samples_split`	8	10
`min_samples_leaf`	3	2
`max_features`	`sqrt`	`None`

5.3.3 XGBoost

The next tree-based ensemble model is the XGBoost classifier. This model serves as a direct point of comparison to the Random Forest. A primary reason for including XGBoost in this study is that it allows for a direct comparison of computational efficiency between CPU and GPU processing. To achieve this, the configuration dictionaries are kept identical for both executions. The only difference is the hardware device parameter, which is set to use the GPU. This controlled setup ensures a precise evaluation of the energy and emissions generated by the exact same model architecture on different hardware. This setup directly addresses the third research question: *How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*.

The break-even analysis described in Section 4.1.1 is computed in the plotting pipeline by reading all HIGGS subset rows from `results.csv` and fitting the log-log model separately for CPU and GPU using `numpy.polyfit`. The intersection is calculated analytically using Equation (4.1).

Break-Even-Wert aus finalem Run eintragen und in Results referenzieren.

Additionally, XGBoost has a specific technical requirement for handling target variables. While the Random Forest algorithm natively adapts to different class structures, XGBoost requires explicit instructions for its learning objective. As mentioned in Section 4.2, the *Wine* dataset consists of three distinct categories. Therefore, the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified number of classes. In contrast, the *Credit* and *Higgs* datasets contain only two categories. Consequently, their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Similar to the previous baseline algorithm, the parameters tailored to each dataset were determined using the Optuna optimization framework. Furthermore, the algorithm processes the unscaled feature matrix X directly. It also uses the globally defined random state parameter to guarantee a deterministic execution across all hardware setups. The resulting configurations used for the final training phase are detailed in Table 5.3.

Table 5.3: Optimized hyperparameters for the XGBoost classifier across all datasets.

Parameter	*Wine*	*Credit*	*Higgs*
`objective`	`multi:softmax`	`binary:logistic`	`binary:logistic`
`eval_metric`	`mlogloss`	`logloss`	`logloss`
`num_class`	3	N/A	N/A
`n_estimators`	464	108	475
`max_depth`	7	3	8
`learning_rate`	0.0139	0.2878	0.2428
`subsample`	0.7790	0.9640	0.7100
`colsample_bytree`	0.7500	0.8430	0.9368
`min_child_weight`	5	7	2

5.3.4 Multilayer Perceptron

As discussed in Section 4.3, a Multilayer Perceptron was selected to represent the neural network family. The implementation uses PyTorch as the primary deep-learning framework, combined with the Skorch library. Skorch wraps the PyTorch model in an interface that is fully compatible with the scikit-learn ecosystem. This is necessary because the entire evaluation logic, including cross-validation and pipeline construction, relies heavily on scikit-learn.

A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven.

The `MLPModule` inherits from `nn.Module` and builds the hidden layers by iterating over the layer size list. Each hidden block consists of `nn.Linear` followed by `nn.BatchNorm1d`, a ReLU activation, and a dropout layer (see Section 2.3.5 for the theoretical background on these components). After all hidden blocks, a single linear output layer maps to the number of target classes with no activation, since `CrossEntropyLoss` expects raw logits.

To minimize the overall training time, the implementation checks for CUDA availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types, and neural networks react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins.

The neural network is embedded within a scikit-learn `Pipeline` together with a

StandardScaler. Since Multilayer Perceptrons lack built-in scale invariance, standardizing the inputs to a mean of zero and a unit variance is necessary to keep the gradient flow stable. The NeuralNetClassifier provided by Skorch receives all architecture parameters through a specific naming convention. It uses the module__* prefix to forward keyword arguments directly to the constructor of the MLPModule. The Adam optimizer and the CrossEntropyLoss criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using torch.manual_seed to guarantee exact reproducibility across all iterations.

The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (skorch.callbacks.EarlyStopping) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

5.4 Hyperparameter Optimization

Building on the design outlined in Section 4.6, this section describes how Optuna was integrated into the evaluation pipeline and which search spaces were defined for each model. The tuning logic is encapsulated in three dedicated scripts (tune_xgb.py, tune_rfc.py, and tune_mlp.py), one per tunable model. All three follow the same structural pattern and differ only in the model that is instantiated and the search space that is queried from the trial object. The target dataset is selected via a command-line argument, and the number of trials is read from the N_TRIALS environment variable, with a default of 50. This separation between configuration and code allows individual tuning runs to be triggered without modifying the source files. The search spaces used across all three tuning scripts are summarized in Table 5.4. Integer and continuous parameters define an inclusive range, while categorical parameters list the discrete choices available to the sampler.

After the dataset has been loaded through the shared load_data() function described in Section 5.2, each script constructs a KFold splitter using the same global constants CV_FOLDS and RANDOM_STATE that are also applied during the final evaluation. Reusing the identical splitter ensures that the F1 score returned to Optuna is computed under the same validation conditions as the metrics reported later in Chapter 6, eliminating any discrepancy between the score the optimizer maximizes and the score the study ultimately reports.

The core of every script is an objective(trial) function that draws a candidate hyperparameter configuration from the trial object, instantiates the corresponding clas-

Table 5.4: Hyperparameter search spaces defined in the Optuna tuning scripts.

Model	Parameter	Type	Range / Values
Random Forest	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 20]
	`min_samples_split`	integer	[2, 20]
	`min_samples_leaf`	integer	[1, 10]
	`max_features`	categorical	{`sqrt`, `log2`, `None`}
XGBoost	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 8]
	`learning_rate`	float (log)	[0.01, 0.3]
	`subsample`	float	[0.6, 1.0]
	`colsample_bytree`	float	[0.6, 1.0]
	`min_child_weight`	integer	[1, 10]
MLP	`n_layers`	integer	[1, 4]
	`layer_i` (per layer)	categorical	{64, 128, 256, 512}
	`dropout_rate`	float	[0.0, 0.5]
	`lr`	float (log)	$[10^{-4}, 10^{-1}]$
	`batch_size`	categorical	{1024, 4096, 8192}

sifier, and returns the mean weighted F1 score obtained from a five-fold cross-validation via `cross_val_score` with `make_scorer(f1_score, average="weighted")`. This is the only output that Optuna is exposed to. Everything else, such as emissions tracking and timing, is handled outside the objective function.

Each study is created via `optuna.create_study(direction="maximize")` and parameterized with a `TPESampler` seeded with `RANDOM_STATE`. Seeding the sampler is what makes the trial sequence reproducible across re-runs. Without it, the suggested hyperparameter combinations would differ on every execution, and the resulting tuning emissions would no longer be comparable. The optimization itself is launched through `study.optimize(objective, n_trials=N_TRIALS)`.

To capture the energy footprint of the tuning process itself, the entire `study.optimize` call is wrapped in a CodeCarbon `EmissionsTracker` with a script-specific `project_name` (e.g. `tune_mlp_higgs`), so that tuning runs are clearly distinguishable from training runs in the `emissions/` directory. The same tracker is used across all three scripts and is configured identically to the trackers used during training, as described in Section 5.5.

Once a study completes, the best score, the corresponding hyperparameter dictionary, the duration, the emissions reported by the tracker, and the trial count are written to a shared `best_params.json` file via the `save_best_params()` utility. The function performs an upsert keyed by model and dataset, so that re-running a single tuning configuration overwrites only its own entry and leaves the other results untouched. The model scripts introduced in Section 5.3 consume this file through the corresponding

`load_best_params()` helper, which decouples tuning from training entirely: the final training pipeline never instantiates Optuna, it merely reads the previously persisted hyperparameter dictionary and passes it to the classifier constructor.

5.5 Emissions Measurement

Measuring energy consumption and CO_2 emissions is the central objective of this study, and the measurement infrastructure therefore required particularly careful design. The following subsections describe how CodeCarbon was integrated into each script, how measurement coverage was defined across model and dataset combinations, and what hardware-specific constraints and limitations apply to the collected values.

5.5.1 CodeCarbon Integration

Energy consumption and CO_2 emissions were tracked using *CodeCarbon* as described in Section 2.2.3. In each model script, an `EmissionsTracker` instance is created before the cross-validation loop begins and configured with two key parameters: `output_dir`, which points to the shared `emissions/` directory at the project root, and `project_name`, which uniquely identifies the run. The tracker is then started by calling `tracker.start()` immediately before the `cross_validate()` call, and stopped via `tracker.stop()` once all folds have completed. The return value of `tracker.stop()` is a single float representing the total CO_2-equivalent emissions in kilograms accumulated over the entire tracked interval. This value is what gets passed to `save_results()` and written to `results.csv`.

The placement of the tracker boundaries was a deliberate choice. Data loading and preprocessing happen before `tracker.start()`, so they are excluded from the measurement. Only the cross-validation itself, which includes all five training and evaluation passes, falls inside the tracked interval. This ensures that the reported emissions value reflects the cost of model training and evaluation exclusively, and that values are directly comparable across models and datasets without interference.

Figure 5.2 illustrates this structure. Prior to training, data is loaded and, where required, features are normalized using `StandardScaler`. Scaling was applied to Logistic Regression and MLP, as these models are sensitive to the magnitude of input features, while tree-based models (Random Forest, XGBoost) were trained on the raw input data.

To allow fine-grained analysis, each model–dataset combination receives its own `EmissionsTracker` instance with a unique `project_name`. The naming convention follows the pattern `<model>_<dataset>`, for example `lr_wine`, `rf_credit`, or `xgb_gpu_higgs`. This scheme makes individual runs immediately identifiable in the `emissions/` directory, where CodeCarbon writes one CSV file per tracked run in addition to returning the

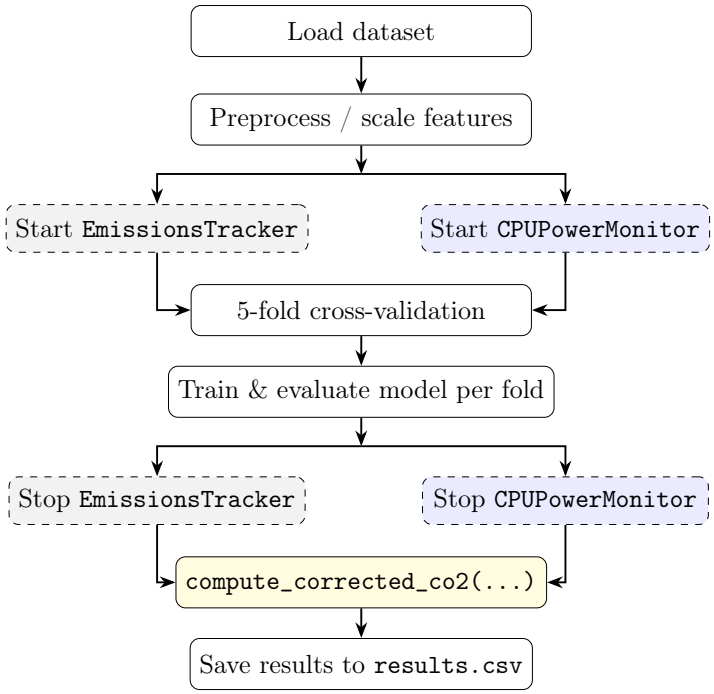

Figure 5.2: Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon `EmissionsTracker` calls; dashed blue boxes indicate `CPUPowerMonitor` calls (background thread, 0.25 s sampling). Both run in parallel during cross-validation. After training, `compute_corrected_co2` replaces CodeCarbon's TDP-based CPU estimate with the hardware sensor value.

scalar value in code. Tuning scripts follow the same pattern preceded by `tune_`, for example `tune_mlp_higgs` or `tune_xgb_cpu_wine`. In total, the experiment produces tracked emissions records for 14 training runs and up to eight tuning runs. The separate CSV files serve as a raw audit trail, while the values extracted via `tracker.stop()` are the primary source for the analysis presented in Chapter 6.

5.5.2 Hardware Measurement and Limitations

The general measurement approach used by CodeCarbon, including direct GPU readout via NVML, RAM estimation per DIMM slot, and TDP-based CPU fallback on Windows, is described in Section 2.2.3. This subsection focuses on the concrete implications for the hardware used in this study.

CodeCarbon's TDP-based CPU estimation on Windows has been replaced in this study by a custom `CPUPowerMonitor` class implemented in `models/power_monitor.py`. It runs in a background thread and polls the CPU Package Power sensor every 0.25 seconds via the *LibreHardwareMonitor* library (`HardwareMonitor` Python package). This library reads the CPU Package Power sensor directly from the hardware, providing true power values rather than a TDP approximation. In practice, CodeCarbon underestimated CPU power by a factor of roughly 9× compared to the hardware sensor (e.g. 6.6 W vs. 57.8 W for Logistic Regression on the Credit dataset). The corrected

CO_2 value in `results.csv` therefore replaces CodeCarbon's CPU contribution with the hardware-measured value, while GPU and RAM energy are still taken from CodeCarbon. The original CodeCarbon total is retained as `co2eq_codecarbon_kg` for reference.

This approach does, however, come with notable limitations. Because LibreHardwareMonitor requires privileged access to hardware sensors, all measurements in this study were conducted with the process running under administrator privileges. Without elevated rights, the `CPUPowerMonitor` fails to retrieve sensor data and the corrected CO_2 estimate cannot be computed.

A second limitation concerns the differences in CPU parallelisation across models. Logistic Regression, for instance, relies on the SAG solver, which executes on a single core, whereas Random Forest and XGBoost exploit multi-threading across all available threads. This disparity directly affects the instantaneous power draw: a single-threaded workload draws considerably fewer watts. However, the reduced wattage is largely offset by a proportionally longer runtime, meaning that the total energy in watt-hours may converge to a similar value. The implications of this trade-off for the comparability of emissions estimates are discussed in Chapter 7.

A secondary limitation is that CodeCarbon measures at the process level, meaning background system activity on Windows 11 contributes to the recorded energy. This effect is expected to be small relative to the dominant training workload but cannot be fully eliminated.

5.6 Inference Latency Measurement

Inference time was measured separately after training, outside the emissions tracker. For each model, a single sample from the test set was passed through the trained model and the elapsed time was recorded using `time.perf_counter()`. This provides a hardware-level estimate of per-sample prediction latency, which is relevant in deployment scenarios where repeated inference can accumulate significant energy costs [39].

Only a single row was passed to the model rather than a full batch. This was intentional: batching would measure throughput rather than the actual time it takes to process one request, which is the more relevant figure in deployment. A single-prediction latency can be extrapolated to real-world usage, where a model may handle millions of requests and the per-sample cost accumulates into a meaningful energy expenditure.

The model used for this measurement is taken from the first cross-validation fold via `cv_results["estimator"][0]`, which is made available by passing `return_estimator=True` to `cross_validate()`. The single-row slice `X[:1]` is then passed to `trained_model.predict()`, with the elapsed time captured using `time.perf_counter()` for sub-millisecond precision.

5.7 Results Persistence and Orchestration

This section describes how experimental results are stored and how the individual model scripts are coordinated into complete experiment runs. The two concerns are closely related: the storage format determines what data is available for analysis, while the orchestration layer controls how and in what order the model scripts are executed.

5.7.1 Results Schema

The results of every model run are written to `results/results.csv`, which serves as the central data source for all subsequent analysis and visualisation. Each row represents one complete model-dataset combination and is appended to the file rather than overwriting it, so that results from multiple runs accumulate without data loss. The file is managed using Python's `csv.DictWriter`, which writes a header row on first creation and appends subsequent rows without repeating the header. A consequence of the append-only design is that re-running a configuration produces duplicate rows, which must be filtered during analysis. Table 5.5 lists all columns.

Table 5.5: Column schema of `results.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`accuracy`	float	Mean CV accuracy
`f1`	float	Mean weighted F1 score
`co2eq_kg`	float	Corrected CO_2
`co2eq_codecarbon_kg`	float	Original CodeCarbon estimate
`cpu_power_hw_w`	float	Average CPU package power (W)
`cpu_energy_hw_wh`	float	Total CPU energy (Wh)
`training_time_s`	float	Total CV wall-clock time (s)

The primary emissions value is `co2eq_kg`, which combines the hardware-sensor CPU reading with CodeCarbon's GPU and RAM estimates as described in Section 5.5.2. The column `co2eq_codecarbon_kg` retains the original CodeCarbon total for reference. The difference between the two columns is examined in Chapter 7.

Inference latency is stored separately in `results/inference_time.csv`, as it is measured outside the emissions tracker and has a different structure. Like `results.csv`, it uses append-only writes via `csv.DictWriter`. Table 5.6 shows its columns.

5.7.2 Experimental Orchestration

Three orchestration scripts coordinate the full experiment. Each model script is launched as an isolated subprocess rather than being imported directly. This is important because

Table 5.6: Column schema of `inference_time.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`inference_time`	float	Single-row prediction latency (s)
`co2eq_kg`	float	Corrected CO_2 of the training run

CodeCarbon operates at the process level: running multiple trackers within the same process risks state leakage between measurements. Subprocess isolation ensures that each tracker starts and stops cleanly. A failed script does not abort the overall run; failures are collected and reported at the end.

`run_all.py` is the main entry point for the full experiment. It runs in two sequential phases. Phase 1 executes all tuning scripts for Random Forest, XGBoost, and MLP across their respective datasets, and saves the best parameters to `best_params.json`. Phase 2 then runs all five model scripts across all three datasets. Each script is launched via `subprocess.run()` with the working directory set to `models/`, so that relative imports resolve correctly.

The other two scripts are used for the subset-based analyses. `run_xgb_breakeven` runs XGBoost CPU and GPU directly on nine HIGGS subset sizes ranging from 1,000 to 11,000,000 rows. Unlike the other orchestration scripts, it calls the model logic directly rather than via subprocess, because it manages the emissions tracker and the cross-validation loop itself. This script provides the data for the break-even analysis in Chapter 6. `run_scaling_all_models.py` runs all models except Random Forest via subprocess across five HIGGS subset sizes (100k to 11M rows), and provides the data for the scaling analysis.

Both subset scripts control the dataset size through an environment variable `TEST_NROWS`. When set, `load_data()` reads this value and draws a stratified sample of that size instead of loading the full dataset. The value is written to the `nrows` column in `results.csv` as a concrete integer, making subset results directly filterable during analysis. The key advantage of this approach is that no changes to any model script are required; the subset size is controlled entirely from the outside.

Chapter 6

Results and Evaluation

Chapter 7

Discussion

Zur Aussagekraft des Break-Even-Points: Der log-log-Fit basiert auf 9 HIGGS-Subset-Größen (1k–11M) mit gleichen Hyperparametern — das isoliert den Dataset-Größen-Effekt sauber. Dennoch Limitierungen benennen: (1) Potenzgesetz ist eine Approximation. (2) Ergebnis gilt nur für diese Hardware und diesen Datensatz. (3) Fit empfindlich gegenüber Ausreißern an den Rändern. Break-Even-Wert als Größenordnung interpretieren, nicht als präzisen Schwellenwert.

CodeCarbon vs. HardwareMonitor comparison: discuss the systematic underestimation of CPU power by CodeCarbon on Windows (TDP-based fallback vs. sensor readout). Show the difference between `co2eq_codecarbon_kg` and `co2eq_kg` per model/dataset. Interpret what this means for reproducibility and comparability of emissions studies that rely solely on CodeCarbon.

Parallelisation trade-off: Logistic Regression (SAG solver, single core) draws fewer watts than Random Forest / XGBoost (multi-threaded), but the longer runtime may result in comparable total energy in Wh. Discuss whether watt-based or Wh-based comparison is the fairer metric for cross-model emissions comparability, and what this means for interpreting the results.

Chapter 8

Conclusion and Outlook

Bibliography

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[2] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.

[3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[4] R. Verdecchia, L. Cruz, J. Sallou, M. Lin, J. Wickenden, and E. Hotellier, "Data-Centric Green AI An Exploratory Empirical Study," in *2022 International Conference on ICT for Sustainability (ICT4S)*, Plovdiv, Bulgaria: IEEE, Jun. 2022, pp. 35–45, ISBN: 978-1-6654-8286-8. DOI: 10.1109/ICT4S55073.2022.00015 Accessed: Mar. 26, 2026.

[5] *Green AI Position Paper*, https://greensoftware.foundation/policy/research/green-ai-position-paper/. Accessed: Mar. 30, 2026.

[6] T. Eilam, P. Bello-Maldonado, B. Bhattacharjee, C. Costa, E. K. Lee, and A. Tantawi, "Towards a Methodology and Framework for AI Sustainability Metrics," in *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, ser. HotCarbon '23, New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 1–7, ISBN: 979-8-4007-0242-6. DOI: 10.1145/3604930.3605715 Accessed: Apr. 10, 2026.

[7] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, *Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning*, Nov. 2022. DOI: 10.48550/arXiv.2002.05651 arXiv: 2002.05651 [cs]. Accessed: Mar. 19, 2026.

[8] J. Dodge et al., "Measuring the Carbon Intensity of AI in Cloud Instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT '22, New York, NY, USA: Association for Computing

Machinery, Jun. 2022, pp. 1877–1894, ISBN: 978-1-4503-9352-2. DOI: 10.1145/3531146.3533234 Accessed: Mar. 11, 2026.

[9] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.

[10] N. Bannour, S. Ghannay, A. Névéol, and A.-L. Ligozat, "Evaluating the carbon footprint of NLP methods: A survey and analysis of existing tools," in *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, Virtual: Association for Computational Linguistics, 2021, pp. 11–21. DOI: 10.18653/v1/2021.sustainlp-1.2 Accessed: Mar. 26, 2026.

[11] L. Lannelongue, J. Grealey, and M. Inouye, "Green Algorithms: Quantifying the Carbon Footprint of Computation," *Advanced Science*, vol. 8, no. 12, p. 2 100 707, 2021, ISSN: 2198-3844. DOI: 10.1002/advs.202100707 Accessed: May 4, 2026.

[12] L. F. W. Anthony, B. Kanding, and R. Selvan, *Carbontracker: Tracking and Predicting the Carbon Footprint of Training Deep Learning Models*, Jul. 2020. DOI: 10.48550/arXiv.2007.03051 arXiv: 2007.03051 [cs]. Accessed: May 4, 2026.

[13] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

[14] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2017, ISBN: 978-0-387-84858-7. Accessed: May 4, 2026.

[15] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 18 932–18 943. Accessed: Mar. 26, 2026.

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.

[17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine.," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1013203451 Accessed: Apr. 19, 2026.

[18] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[19] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[20] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, Mar. 1986, ISSN: 1573-0565. DOI: 10.1007/BF00116251 Accessed: Apr. 29, 2026.

[21] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" *Advances in Neural Information Processing Systems*, vol. 35, pp. 507–520, Dec. 2022. Accessed: Apr. 29, 2026.

[22] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[23] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, Aug. 1996, ISSN: 1573-0565. DOI: 10.1007/BF00058655 Accessed: Apr. 29, 2026.

[24] E. Scornet, G. Biau, and J.-P. Vert, "Consistency of Random Forests," *The Annals of Statistics*, vol. 43, Apr. 2014. DOI: 10.1214/15-AOS1321

[25] G. Biau and E. Scornet, "A random forest guided tour," *TEST*, vol. 25, no. 2, pp. 197–227, Jun. 2016, ISSN: 1863-8260. DOI: 10.1007/s11749-016-0481-7 Accessed: Apr. 29, 2026.

[26] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[27] R. Mitchell and E. Frank, "Accelerating the XGBoost algorithm using GPU computing," *PeerJ Computer Science*, vol. 3, e127, Jul. 2017, ISSN: 2376-5992. DOI: 10.7717/peerj-cs.127 Accessed: Apr. 29, 2026.

[28] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, ISSN: 0893-6080. DOI: 10.1016/0893-6080(89)90020-8 Accessed: Apr. 29, 2026.

[29] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Apr. 19, 2026.

[30] D. P. Kingma and L. J. Ba, "Adam: A Method for Stochastic Optimization," 2015. Accessed: Apr. 19, 2026.

[31] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, Mar. 1994, ISSN: 1941-0093. DOI: 10.1109/72.279181 Accessed: Apr. 29, 2026.

[32] G. Hinton, V. Nair, and Y. Rachmad, "Rectified Linear Units Improve Restricted Boltzmann Machines Vinod Nair," vol. 27, pp. 807–814, Jun. 2010.

[33] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.

[34] L. Prechelt, "Early Stopping — But When?" In *Neural Networks: Tricks of the Trade: Second Edition*, G. Montavon, G. B. Orr, and K.-R. Müller, Eds., Berlin, Heidelberg: Springer, 2012, pp. 53–67, ISBN: 978-3-642-35289-8. DOI: 10.1007/978-3-642-35289-8_5 Accessed: Apr. 29, 2026.

[35] J. Nickolls and W. J. Dally, "The GPU Computing Era," *IEEE Micro*, vol. 30, no. 2, pp. 56–69, Mar. 2010, ISSN: 1937-4143. DOI: 10.1109/MM.2010.41 Accessed: May 4, 2026.

[36] P. Koshute, J. Zook, and I. McCulloh, *Recommending Training Set Sizes for Classification*, https://arxiv.org/abs/2102.09382v1, Feb. 2021. Accessed: Mar. 25, 2026.

[37] A. Althnian et al., "Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain," *Applied Sciences*, vol. 11, no. 2, p. 796, Jan. 2021, ISSN: 2076-3417. DOI: 10.3390/app11020796 Accessed: Apr. 16, 2026.

[38] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, Jul. 2009, ISSN: 0306-4573. DOI: 10.1016/j.ipm.2009.03.002 Accessed: May 4, 2026.

[39] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[40] L. Yang and A. Shami, "On hyperparameter optimization of machine learning algorithms: Theory and practice," *Neurocomputing*, vol. 415, pp. 295–316, Nov. 2020, ISSN: 0925-2312. DOI: 10.1016/j.neucom.2020.07.061 Accessed: May 4, 2026.

[41] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, ISSN: 1533-7928. Accessed: Apr. 19, 2026.

[42] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011. Accessed: Apr. 19, 2026.

[43] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[44] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.

[45] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.

[46] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.

[47] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.

[48] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.

[49] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.

[50] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[51] *Claude Code*, Anthropic, 2026.

List of Figures

List of Tables